

FORMATION EMBARQUÉE

Module 04 — VHDL & FPGA

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 04 — VHDL & FPGA : décrire du matériel, pas écrire du logiciel

⚠ Changement de paradigme. Le VHDL **n'est pas un langage de programmation** : c'est un langage de **description de matériel** (HDL). Ton code ne s'« exécute » pas ligne par ligne — il est **synthétisé** en portes logiques et bascules qui fonctionnent toutes **en parallèle, tout le temps**. Oublie la boucle, pense circuit.

1. FPGA : c'est quoi ?

Un **FPGA** (Field-Programmable Gate Array) est une puce contenant des milliers de blocs logiques configurables (LUT + bascules), reliés par un réseau d'interconnexions programmable. On y « charge » un circuit décrit en VHDL ou Verilog.

	Microcontrôleur	FPGA
Exécution	séquentielle (1 instruction à la fois)	parallèle (tout à chaque front d'horloge)
Latence	variable (interruptions, pipeline)	déterministe au cycle près
Fort pour	logique applicative, coût	traitement vidéo/signal, protocoles rapides, temps réel dur
Modifiable	reflasher le programme	recharger le <i>bitstream</i> (le circuit change !)

Outils et matériel

- **Simulation gratuite sans carte** : **GHDL** + **GTKWave** (open source), ou <https://www.edaplayground.com> dans le navigateur. **Commence par là.**
- **Synthèse** : AMD/Xilinx **Vivado** (cartes Basys 3, Arty), Intel/Altera **Quartus** (DE10-Lite), Gowin EDA (Tang Nano 9K à ~15 €), ou le flot open source Yosys/nextpnr (Lattice iCE40/ECP5).
- Flot de travail : écrire → **simuler** (testbench) → synthétiser → placer-router → vérifier le timing → charger le bitstream.

2. Structure d'un fichier VHDL

Tout module a deux parties : l'**entité** (la boîte : nom + broches) et l'**architecture** (ce qu'il y a dedans).

```

library ieee;
use ieee.std_logic_1164.all;      -- types std_logic / std_logic_vector
use ieee.numeric_std.all;         -- unsigned/signed et arithmétique

entity porte_et is
  port (
    a : in  std_logic;           -- entrée 1 bit
    b : in  std_logic;
    s : out std_logic           -- sortie
  );
end entity;

architecture rtl of porte_et is
begin
  s <= a and b;                  -- affectation CONCURRENTE : du fil + une porte
end architecture;

```

Points de syntaxe : - Insensible à la casse ; commentaires avec `--` . - `<=` = affectation de **signal** (≠ `:=` pour les variables). - `rtl` (Register Transfer Level) est un nom d'architecture conventionnel.

2.1 Les types essentiels

```

signal bit_seul : std_logic;      -- '0', '1', 'Z'(haute imp.), '- '...
signal bus8     : std_logic_vector(7 downto 0); -- 8 fils, bit 7 = MSB
signal compteur : unsigned(15 downto 0); -- vecteur qui sait compter
signal temp     : signed(11 downto 0); -- signé (complément à 2)
signal n        : integer range 0 to 99; -- pour compteurs bornés

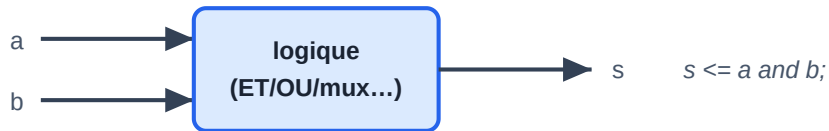
bus8 <= "10100101";              -- littéral binaire
bus8 <= x"A5";                   -- littéral hexa
compteur <= compteur + 1;         -- OK sur unsigned (pas sur std_logic_vector)
bus8 <= std_logic_vector(compteur(7 downto 0)); -- conversions explicites

```

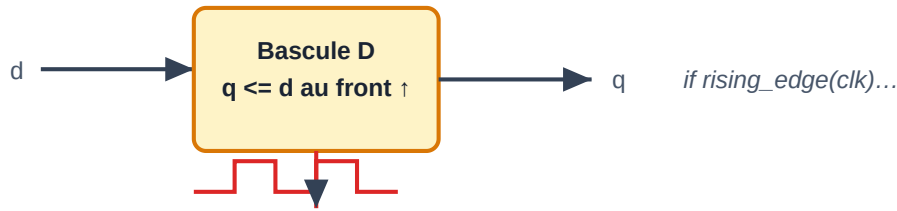
Utilise `unsigned` / `signed` (de `numeric_std`) pour tout ce qui compte, `std_logic_vector` pour les bus « neutres » aux interfaces.

3. Concurrent vs séquentiel : le cœur du VHDL

Combinatoire — la sortie ne dépend QUE des entrées (pas de mémoire)



Séquentiel — la sortie dépend aussi du PASSÉ (registre cadencé)



Logique combinatoire (sans mémoire) contre séquentielle (registre cadencé)

3.1 Instructions concurrentes (hors process)

Chaque ligne décrit un morceau de circuit **permanent et simultané** — l'ordre des lignes ne compte pas :

```
s1 <= a and b;
s2 <= not c;
y <= s1 or s2;                                -- trois "morceaux de circuit" en parallèle

-- Multiplexeur : selon la valeur de sel
with sel select
    y <= a when "00",
        b when "01",
        c when others;

-- Conditionnelle concurrente
y <= a when (en = '1') else '0';
```

3.2 Le process : décrire un bloc en style séquentiel

Un **process** contient du code lu séquentiellement *par le simulateur/ synthétiseur* pour en déduire un circuit.

Deux formes canoniques :

Combinatoire (sensible à toutes ses entrées) :

```
process (a, b, sel)                            -- liste de sensibilité : TOUTES les entrées lues
begin
    if sel = '1' then
        y <= a;
    else
        y <= b;
    end if;                                    -- chaque signal affecté doit l'être dans
end process;                                  -- TOUS les chemins, sinon → latch involontaire !
```

Séquentiel / synchrone (sensible à l'horloge) — c'est un **registre** :

```

process (clk)
begin
    if rising_edge(clk) then          -- au front montant...
        if reset = '1' then
            q <= (others => '0');    -- reset synchrone
        else
            q <= d;                  -- une bascule D par bit
        end if;
    end if;
end process;

```

3.3 Les deux règles qui évitent 90 % des bugs de débutant

1. **Un signal n'est affecté que dans UN seul process** (sinon : conflit, « multiple drivers »).
2. Dans un process combinatoire, **tout signal de sortie reçoit une valeur dans toutes les branches** (mettre une affectation par défaut en tête du process), sinon le synthétiseur crée un **latch** — presque toujours un bug.

3.4 Signaux vs variables

- **signal** (`<=`) : prend sa nouvelle valeur **à la fin du delta-cycle** — dans un process synchrone, la lecture voit encore l'ancienne valeur. C'est ce qui modélise fidèlement une bascule.
 - **variable** (`:=` , locale au process) : mise à jour **immédiate**. Utile pour du calcul intermédiaire combinatoire dans un process.
-

4. Les briques de base à savoir écrire les yeux fermés

4.1 Compteur avec division d'horloge (faire clignoter une LED)

```
entity blink is
    generic ( F_CLK : natural := 50_000_000 );          -- paramètre : 50 MHz
    port ( clk : in std_logic; led : out std_logic );
end entity;

architecture rtl of blink is
    signal cpt      : unsigned(25 downto 0) := (others => '0');
    signal led_int  : std_logic := '0';
begin
    process (clk)
    begin
        if rising_edge(clk) then
            if cpt = F_CLK/2 - 1 then                    -- toutes les 0,5 s
                cpt      <= (others => '0');
                led_int <= not led_int;
            else
                cpt <= cpt + 1;
            end if;
        end if;
    end process;
    led <= led_int;
end architecture;
```

Note : **pas de `delay()` possible** — attendre = compter des fronts d'horloge.

4.2 Registre à décalage, détecteur de front

```
-- Détecteur de front montant sur une entrée (déjà synchrone)
process (clk)
begin
    if rising_edge(clk) then
        btn_prec <= btn;
    end if;
end process;
front <= btn and not btn_prec;      -- '1' pendant exactement 1 cycle
```

Toute entrée **asynchrone** (bouton, signal externe) doit d'abord passer par **2 bascules en série** (synchroniseur) pour éviter la métastabilité.

4.3 Machine d'états finis (FSM) — le motif central

```
architecture rtl of feu_tricolore is
    type t_etat is (VERT, ORANGE, ROUGE);
    signal etat : t_etat := ROUGE;
    signal cpt : unsigned(27 downto 0) := (others => '0');
begin
    process (clk)
    begin
        if rising_edge(clk) then
            cpt <= cpt + 1;
            case etat is
                when VERT =>
                    if cpt = DUREE_VERT then
                        etat <= ORANGE; cpt <= (others => '0');
                    end if;
                when ORANGE =>
                    if cpt = DUREE_ORANGE then
                        etat <= ROUGE; cpt <= (others => '0');
                    end if;
                when ROUGE =>
                    if cpt = DUREE_ROUGE then
                        etat <= VERT; cpt <= (others => '0');
                    end if;
            end case;
        end if;
    end process;

    -- Sorties décodées de l'état (style Moore : sorties = f(état))
    feu_vert  <= '1' when etat = VERT  else '0';
    feu_orange <= '1' when etat = ORANGE else '0';
    feu_rouge  <= '1' when etat = ROUGE  else '0';
end architecture;
```

Compare avec la FSM en C du module 01 : même concept, mais ici l'« évaluation » a lieu à chaque front d'horloge, physiquement.

5. Hiérarchie : instancier des composants

On assemble des modules comme des puces sur une carte :

```

architecture rtl of top is
    signal tick : std_logic;
begin
    u_div : entity work.diviseur          -- instantiation directe
        generic map ( RATIO => 50_000_000 )
        port map ( clk => clk, tick => tick );

    u_fsm : entity work.feu_tricolore
        port map ( clk => clk, en => tick,
                    feu_vert => led(0), feu_orange => led(1), feu_rouge => led(2) );
end architecture;

```

generic = paramètre de compilation (largeur de bus, fréquence...) → modules réutilisables.

6. Le testbench : simuler AVANT de synthétiser

Règle d'or : **on ne charge jamais sur carte un module non simulé**. Un testbench est un module VHDL sans ports qui instancie le module à tester (DUT), génère horloge et stimuli, et vérifie les sorties.

```

library ieee;
use ieee.std_logic_1164.all;

entity tb_porte_et is end entity;          -- pas de ports

architecture sim of tb_porte_et is
    signal a, b, s : std_logic := '0';
begin
    dut : entity work.porte_et port map (a => a, b => b, s => s);

    stim : process
    begin
        a <= '0'; b <= '0'; wait for 10 ns;
        assert s = '0' report "0 and 0 devrait donner 0" severity error;
        a <= '1'; b <= '0'; wait for 10 ns;
        assert s = '0' severity error;
        a <= '1'; b <= '1'; wait for 10 ns;
        assert s = '1' severity error;
        report "Test termine";
        wait;                                -- stoppe le process
    end process;
end architecture;

```

Avec GHDL :

```

ghdl -a porte_et.vhd tb_porte_et.vhd  # analyse
ghdl -e tb_porte_et                  # élaboration
ghdl -r tb_porte_et --wave=ondes.ghw  # simulation
gtkwave ondes.ghw                    # visualiser les chronogrammes

```


`wait for 10 ns`, boucles de stimulation, `report` ... existent **en simulation seulement** — non synthétisables. Apprendre à lire un **chronogramme** dans GTKWave est une compétence à part entière.

7. Synthétisable ou pas ?

Synthétisable ✓	Simulation seulement ✗
<code>if/case</code> , opérateurs logiques et arithmétiques	<code>wait for 10 ns</code>
process avec <code>rising_edge(clk)</code>	<code>after 5 ns</code> dans les affectations
<code>for ... generate</code> , boucles bornées (déroulées)	fichiers, <code>report</code> (ignoré)
division par puissance de 2	division/modulo quelconques (coûteux/refusés)

Toujours garder en tête : « quel circuit suis-je en train de décrire ? » Si tu ne peux pas le dessiner (portes, mux, registres), le synthétiseur non plus.

8. Notions de timing (aperçu)

Entre deux bascules, le signal traverse de la logique combinatoire : ce chemin doit être plus court qu'une période d'horloge (moins la marge). Les outils font une **analyse statique de timing (STA)** et te disent la fréquence max. Tu declares tes horloges dans un fichier de **contraintes** (`.xdc` chez Xilinx, `.sdc` chez Intel/Altera) qui fixe aussi le brochage :

```
# extrait .xdc (Basys 3)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.0 [get_ports clk] ;# 100 MHz
```

9. VHDL vs Verilog

Les deux décrivent la même chose. VHDL : verbeux, fortement typé, dominant en Europe (aéronautique, défense, écoles françaises). Verilog/SystemVerilog : concis, dominant aux USA et dans le monde ASIC. Apprends VHDL à fond ; lire du Verilog viendra tout seul ensuite.

Exercices (fais-les tous sur GHDL/EDA Playground)

1. Écris et teste un **multiplexeur 4 vers 1** (2 bits de sélection) — version concurrente et version process.
2. **Compteur BCD 0–9** avec sortie 7 segments (décodeur combinatoire) ; testbench qui vérifie les 10 motifs.
3. **Anti-rebond matériel** : synchroniseur 2 bascules + compteur qui exige 20 ms de stabilité — puis détecteur de front.

4. **Feu tricolore** ci-dessus, enrichi d'un bouton piéton (le vert passe à l'orange au plus tard 1 s après l'appui).
5. **Émetteur UART 115200 bauds** : FSM start/8 données/stop + compteur de « baud tick ». Vérifie la trame au chronogramme. (Projet sérieux — c'est le « hello world » avancé du FPGA.)

 Suite : [Module 05 — Java](#) ou [Module 06 — Autres langages](#).